Scoping and Naming

CS 1622

Jarrett Billingsley

Class Announcements

- how's project 2 coming along (:

Symbol Tables and Scope Trees

Scoping it out

- the first phase of semantic analysis is **name-checking,** which:
  - **matches names** to the things they refer to; and
  - **checks for scoping violations.**
    - remember that the **scope** of a name is *where* it can be seen.
- **so how do we *do* this stuff?**
  - well, like many other things we've talked about, it's about **formalizing our intuition:** taking rules that we kind of know already, and turning them into an algorithm.
- we also need a **data structure or two…**
  - we need to remember **where each name is declared**
  - and we need some **mapping** from names to referents
- so let's start with…

Symbol tables

- a **symbol table** maps from names to the "things" they refer to.
  - **symbol** means **anything that can have a name:** variables, functions, classes, modules, packages, interfaces, macros, etc. etc.
- as you declare things, they are added to the symbol table.

```
class A {
  static int x = 5;
  static void main() {
    foo(x);
  }
  static void foo(int y) {
    S.o.println(y);
  }
}
```

Name	Referent
"A"	<class A>
"x"	<int x>
"main"	<void main()>
"foo"	<void foo()>
"y"	<int y>

but you can't see **y** from **main**... so should it be in this table?

Instead of ONE symbol table...

- we **nest** the symbol tables in a tree. there's one for each scope.

```
class A {
  static int x = 5;
  static void main() {
    foo(x);
  }
  static void foo(int y) {
    S.o.println(y);
  }
}
```

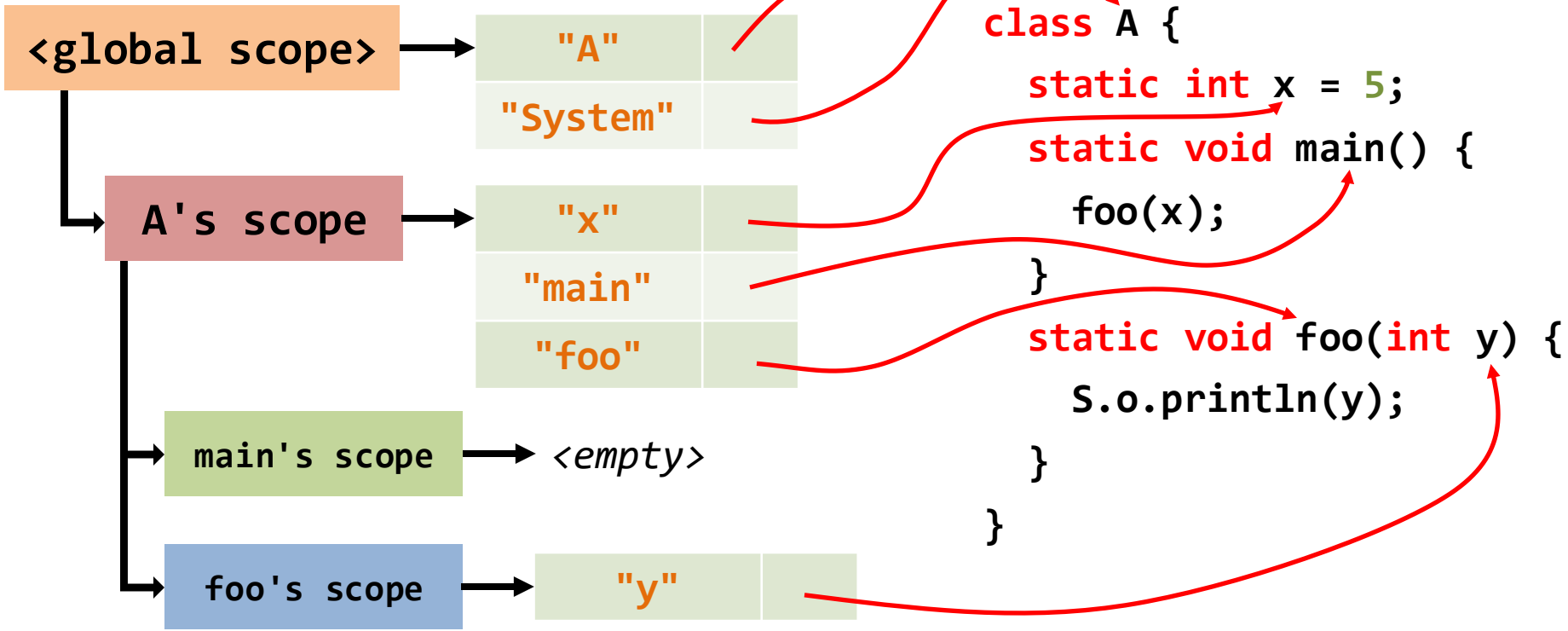
this outermost (global) scope is where e.g. **`System`** lives. **`A`** is inserted there too.

the class **`A`** gets a scope to hold its members, like its variables and methods. this scope is a **child** of the global scope.

then each method gets its own scope, both of which are **children** of **`A`**'s scope.

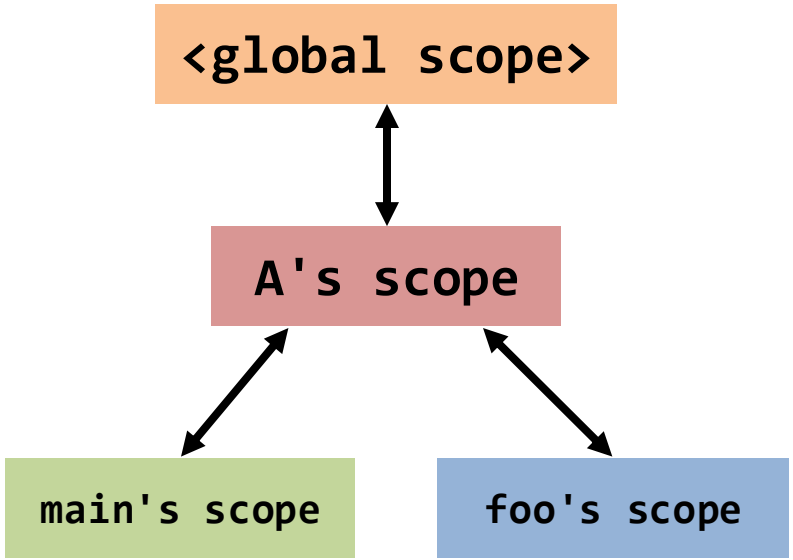
The scope tree data structure

- it's a **tree**, where each node is a **scope**, which has a **symbol table**.
- the symbol tables map from **names** to the **nodes in the AST which define those names.***



Parent scopes

- each child scope **also** needs to know which scope is its **parent**.
  - (this is used in the name resolution algorithm.)
- that means we've got **bidirectional links** between the scopes.



this is going to have *implications* for the way we implement this data structure in Rust...

but then each scope has a symbol table which somehow points *into* the AST? how do we deal with that?

maybe we could integrate this *into* the AST??

WELL, NO.....

Syntax, not semantics

- since the scopes *more or less* follow the syntax ({} ≅ scope)…
  - we might be tempted to jam the scope tree into the AST somehow.
  - **this is a really bad idea.**
- the AST is an abstract ***syntax*** tree, but we're moving ***beyond syntax.***
  - the relationships established by name resolution can crisscross the whole program and form **any shape of graph, including cycles.**
  - and as we get further into semantic analysis – and then into optimization and code generation – the syntax will become less and less relevant.
- so, the scope tree data structure should exist *in parallel to* the AST.
  - we'll represent the links between the two data structures in an indirect way, as we'll see…

Name Resolution

time check ≤ 20

Declaration vs. use

- when we **declare** classes, variables, functions etc. that makes the name **available for use** by other pieces of code.
  - a declaration essentially **inserts a name** into a symbol table.

```
int x = 0;
x = x + 1;
println(x);
x = f();
return x;
```

this is the **declaration** of **x**. it puts **x** into this scope's symbol table.

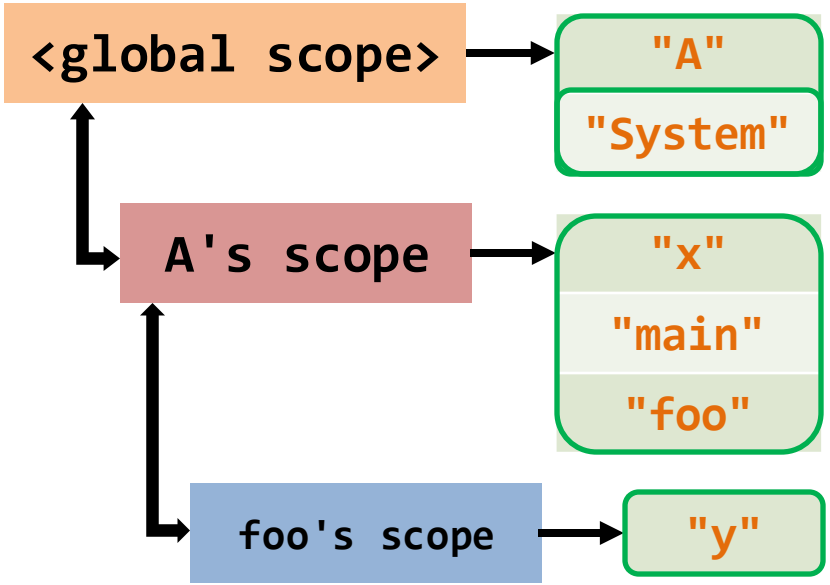
all of these other places where we **refer to x by name** are called **uses.**

for each of these uses, we have to perform **name resolution:** determining **which symbol** the use refers to.

the scope tree/symbol tables and the scoping rules are what we'll use to implement name resolution!

Climbing the tree (animated)

- the name resolution algorithm is actually pretty straightforward.



3. go to the parent scope of that one. is **System** there?

yes! **name resolved.**

2. go to the **foo**'s **parent scope.** is **System** there?

still no?

1. is **System** declared in **foo**'s scope?

no?

```
static void foo(int y) {
  System.out.println(y);
}
```

this use of **System** appears in **foo**'s scope. let's start there.

What if it fails?

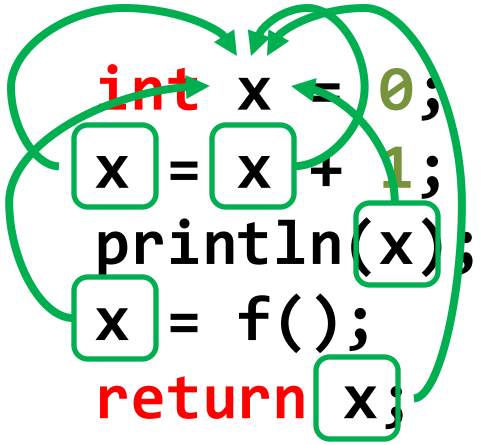
- what if we get up to the global scope and there's no match?
  - well, that's an **error.** that's when your compiler says "unresolved symbol" or "undefined name" or whatever.
- but it could "fail" in another way:

```
int x = System + 5;
```

  - assuming I didn't declare a variable named System...
  - this is nonsense. how do you add a class and an integer?
- well, **don't worry about it.** not at this stage, anyway.
  - name resolution **only cares about matching names to the things that declared them.**
  - this example is a **type error** and will be caught further on during a subsequent phase of semantic analysis.

Keeping track of resolved names

- we *could* perform name resolution every time we need to know the referent during subsequent compiler phases.
  - but real-world experience (by other people) has shown that these resolutions happen *a lot* and that can waste *a lot of time*.
- since the referent never changes, it makes sense to **remember that.**



we can do this by keeping a **map** which maps from the AST nodes which **use** a name to the node which **declares** it. this is *yet another* data structure that I call the **use map**.

it's a bit like drawing arrows all over the code.

again, we'll see how to implement this soon…

Accessing names *inside* things

- this piece of code presents another interesting scenario:

```
System.out.println(y);
```

is this also a name resolution??

well, **yes!** this is saying "access **out** from the **System** scope."

but there's a complication: we may not be able to perform the resolution **until after we do typechecking:**

```
obj.x = 10;
```

here, we have to know **what *type* `obj` is** to know if **x** is a valid name inside of that type.

then we will have to check that the type of **x** is actually valid for this assignment! wow! very circular!!

fortunately this is a *relatively* limited situation that can be resolved later...

Forward/mutual references

- a **forward reference** is when the *use* of a name occurs **before its declaration** in the source code.

```
class A { B b; }
class B {}
```

this is a forward reference, because `B` is declared later.

if we did all the name stuff in **one pass,** we would get an error here because B doesn't exist in the symbol table yet!

this means we have to do name stuff in two passes:

on the first pass, we build the **scope tree** and **insert declarations** into each scope's symbol table;

then on the *second* pass, we can do the **name resolution** using the already-built symbol tables.

...but that doesn't work for locals

- consider this code:

```
static void func() {
  S.o.println(x);
  int x = 10;
}
```

you *know* that this is wrong. but *why?*

the code in a function **executes in order, including local variable declarations.** so on the first line, **x has no value yet.**

there are two ways to solve this:

1. introduce a **new scope** that begins after each local declaration and ends at the enclosing close-brace; or
2. do the name resolution **at the same time** as scope-building, but **only for the code within functions.**

Annotating the AST

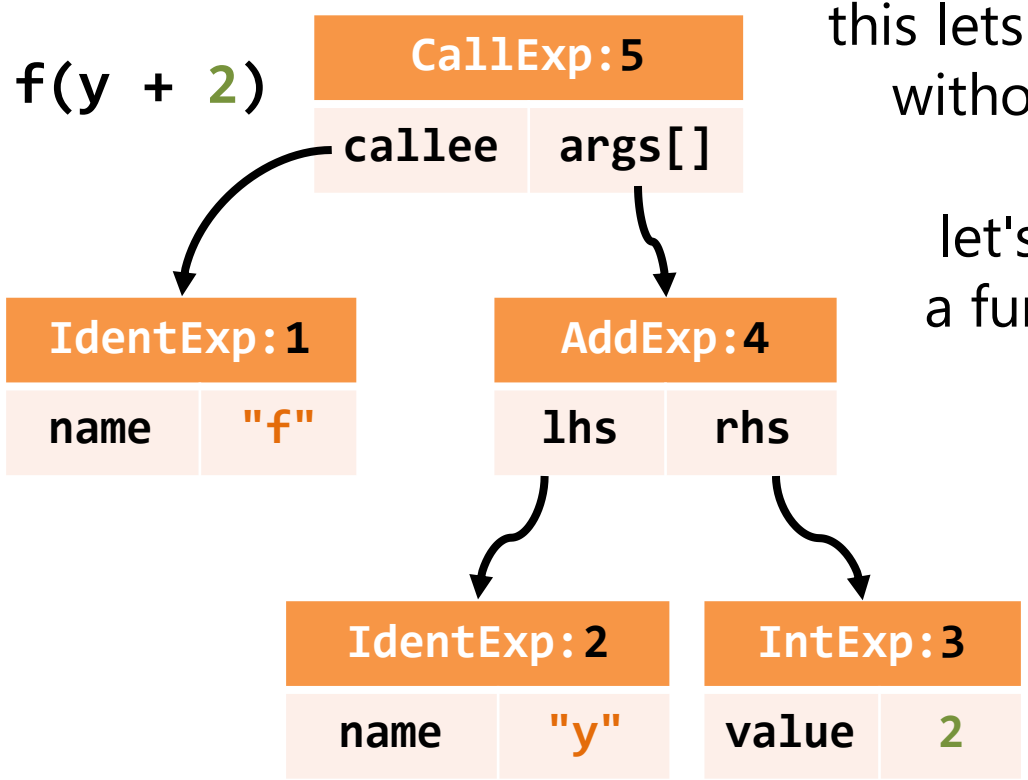
time check ≤ 55

Throwing arrows all over the place

- we've now got **two things** that want to draw arrows all over the AST:
  - **symbol tables** want to point at declarations
  - **name resolution** wants to make AST nodes point at other ones
- in a language like Java, "arrows" are **object reference variables,** and we might be tempted to start throwing those *everywhere*.
  - "add a field to **Identifier** AST nodes so they can point at the declarations they refer to!"
  - "add fields to **{ Block Statement }** AST nodes to hold scope info!"
- OOP makes it really easy to start doing this…
  - but it's **very difficult to stop once you've started,** because it creates **tight coupling** between the two "ends" of the arrow.
- since we'll only be adding more info from now on (typechecking!), it makes sense to reconsider how we "add information" to the AST.

Nametags, not arrows

- first, let's make a small change to our AST.
- **every AST node** is given a unique identifier when it is created.
  - this can be as simple as an int that's incremented for each node.



this lets us **create relationships** without adding any arrows.

let's say the name **f** resolves to a function declared by node **49**.

then in our name resolution map, we add an entry with a key of **1** and a value of **49**.

Wh.. but... why? Why not just point to the node?

- **decoupling** is good practice. it gives you a cleaner design.
- we can add **arbitrarily many** pieces of data to an AST node, without having to change the AST node types at all!
  - we just add more maps that use the node IDs as keys.
  - this lets us **add new compilation passes** and **modularize** the compiler much more easily.
- it improves compiler compile times by not requiring a recompile of the *entire compiler* every time you change the AST type.
  - and real compilers can be ***massive,*** so this is a big timesaver!
- it can improve performance by exploiting **physical locality.**
  - the **cache** in your CPU likes arrays and small structs.
  - the AST node struct stays **small,** meaning it'll likely fit into cache.
  - associated data can even be stored in a **contiguous array.**

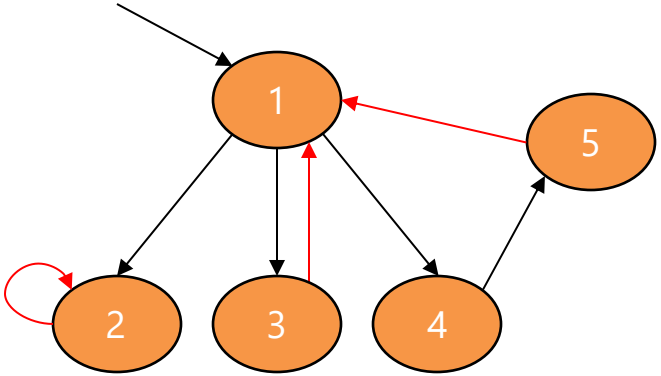
Another reason

- we are also nudged towards this design by Rust.

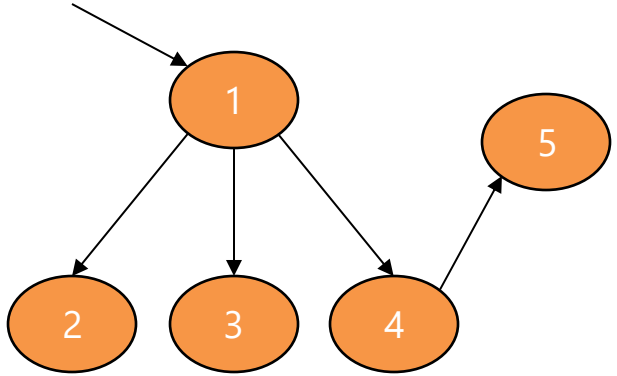
in Java, each object reference is an arrow, and there are **no restrictions** on them.

in Rust, there are multiple kinds of pointers, but **Box cannot be used to make cycles.**

```
class Node { Node[] others; }
```




```
struct Node { others: Vec<Box<Node>> }
```



this "no cycles allowed" thing is a *central part of Rust's ownership, borrowing, and memory safety semantics.*

What??!? You can't have cycles in Rust?

- well, you *can*, but it's discouraged by the language and libraries.
  - **Rc<T>** is another kind of pointer that, with some care, *can* let you represent *certain* kinds of cyclical data structures.
- it's not the Rust designer's fault that they made it harder to represent cyclical data structures.
  - **it's that cyclical data structures are just weird,** and a lot of other languages don't really make that fact obvious.
  - they either hide it, or don't give you the tools to manage it!
- this restriction *does* give Rust some great advantages in memory safety and multithreaded code correctness!
  - which is kind of Rust's whole *deal*
- but it does mean you have to stretch your mind a little.
  - and interestingly, many of the solutions around this restriction end up being faster *and* easier to reason about.

Implementation Details

time check ≤ 85

Well Actually

- *in practice,* it's a good idea to represent **symbols** as objects.

```
class A {
  static int x = 5;
  static void main() {
    foo(x);
  }
  static void foo(int y) {
    S.o.println(y);
  }
}
```

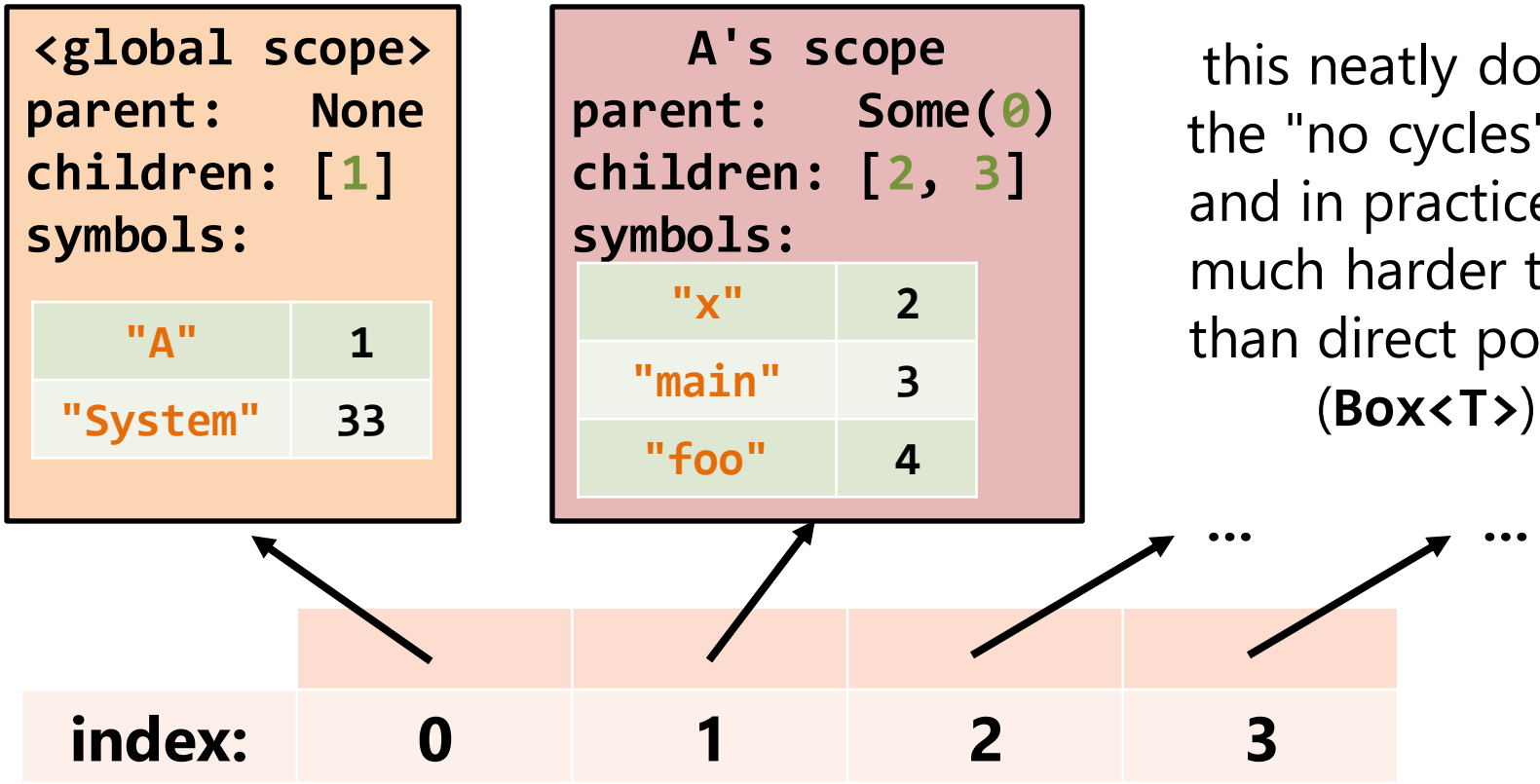
ID	AST	Scope	Name	Kind	...
1	3	0	A	class	...
2	7	1	x	static var	...
3	12	1	main	static func	...
4	22	1	foo	static func	...
5	24	3	y	local var	...
...	...	...	...	...	...

every name declared in the program gets its own symbol object with a unique **symbol ID.**

extracting this info from the AST is time-consuming, so we only do it once. *these* are what the symbol tables *actually* refer to.

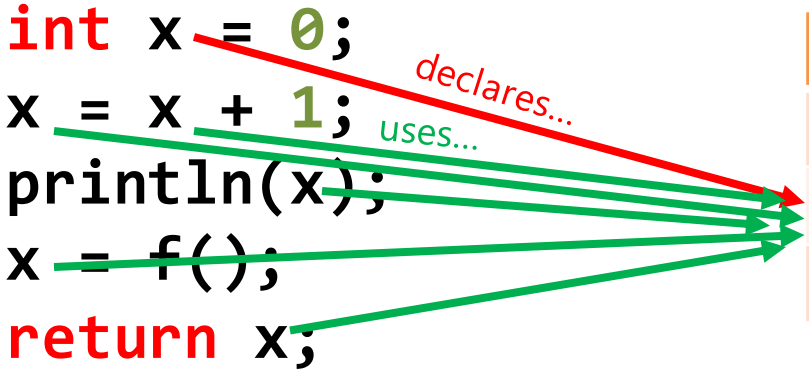
Bidirectionally linked scopes

- an easy way to solve this is to **keep all scopes in a vector,** and have them **refer to each other by their *index* into the vector.**



The declaration and use maps

- since we now represent symbols as their own "thing," the **declaration map** maps from **AST nodes that declare a symbol,** to that symbol.
- the **use map** maps from **AST nodes** to the symbol they use.



ID	AST	Scope	Name	Kind	...
...	...	...	...	...	...
17	54	3	x	local var	...
...	...	...	...	...	...

the actual maps map from **Node IDs** to **Symbol IDs**.

Decls

AST	Sym
74	17
...	...

Uses

AST	Sym
76	17
78	17
...	...

Bundling it all up

- the **symbols, scope tree, declaration map,** and **use map** are all packaged into one big object, the **name context.**
- this can be used in later phases of the compiler!



each phase of semantic analysis generates **more information about the program** which later phases use.

for now, the AST will remain the central representation of the program, but that may change later in compilation…

Building the scope tree and symbol tables

- the actual algorithm for building the scope tree is surprisingly simple.
- the idea is like this:
  - we recursively visit each node in the AST.
  - if we take a **function declaration** node as an example…
    - it will **create a new scope as a child of the current scope…**
    - then **visit its children** (arguments and code) within that scope.
  - something similar happens for **{** Block Statements **}**, classes etc.
  - and when we see a **declaration AST node,** we insert a symbol into the current scope.
    - this also inserts an entry into the **declaration map.**
- and that's about it!
  - there's the "forward reference" thing we talked about, and the stuff about local variables, but those are just tweaks to this algorithm.

Name resolution and building the use map

- finally, name resolution is another recursive AST visit.
- if this is done **simultaneously** with building the scope tree…
  - it's very simple: whenever you see an Identifier AST node, you perform name resolution starting at the current scope.
  - if the name exists, **add an entry to the use map.**
  - if it doesn't, **it's an error.**
- if you built the scope tree in one pass and then do name resolution in a **second pass after it…**
  - there's some bookkeeping to make sure you *start* the name resolution in the **correct scope,** but it's otherwise the same.
- and at the end, you now have all your data structures built and name-checking is complete!
  - name-checking is the easiest semantic analysis pass, and things only get harder from here (: